

Bootcamp: Full Stack Junior

Modulo 1: Gestión Integral de Bases de Datos

Semana 3

Contenido

1

Optimización de consultas

2

Índices

3

Manejo de grandes volúmenes de datos

4

Implementación práctica con MySQL

5

Introducción a bases de datos NoSQL

Contenido:

1. Optimización y Big Data en MySQL

La clave para que una base de datos relacional puede manejar grandes volúmenes de datos es crucial. La optimización no es solo "hacer consultas", sino diseñar para el rendimiento.

- **Indexación Avanzada:** Uso de índices B-Tree y Hash para acelerar la búsqueda en datasets de millones de registros.
- **Particionamiento de Tablas:** Técnica de dividir tablas grandes en partes más pequeñas y manejables sin cambiar la lógica de la aplicación.
- **Query Profiling:** Herramientas para analizar el tiempo de ejecución y cuellos de botella en sentencias complejas.

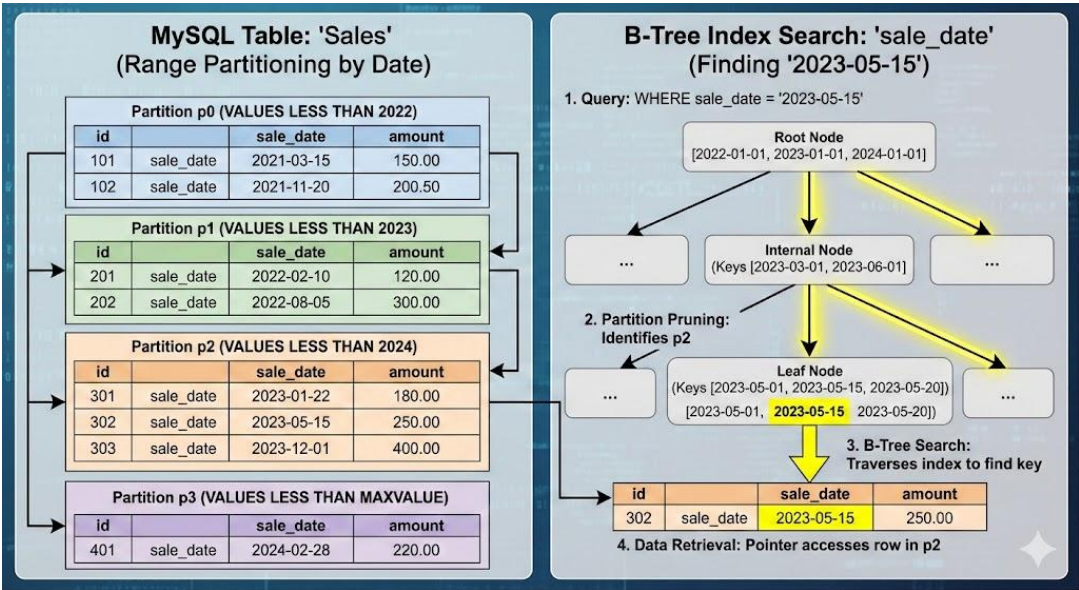


Ilustración 1 –Pariciones de tablas en SQL - Fuente: Gemini Nano Banana Pro

2. Manipulación de tipos JSON en MySQL

MySQL ha evolucionado para permitir esquemas híbridos. El tipo de datos **JSON** permite almacenar datos semi-estructurados dentro de una tabla relacional, combinando la rigidez del esquema con la flexibilidad documental.

2.1. Cuando es necesario usar datos semi-estructurados

En aplicaciones web, móviles y microservicios, a menudo se envían y reciben datos en formato JSON. Soportar JSON nativo permite a MySQL guardar esos datos sin tener que "encajarlos" en tablas rígidas, manteniendo la flexibilidad de un documento y la potencia de SQL.

2.2. Características principales

MySQL valida automáticamente que los datos insertados sean JSON válidos antes de almacenarlos, rechazando formatos incorrectos para garantizar integridad. Internamente, se guarda en formato binario optimizado (similar a LONGBLOB), lo que acelera las consultas en comparación con columnas TEXT tradicionales. Este tipo soporta objetos, arrays, strings, números y valores nulos según el estándar RFC 8259.

Funciones clave para el desarrollador:

- **JSON_EXTRACT():** Extrae datos de un documento JSON.
- **JSON_INSERT() / JSON_REPLACE():** Modifica partes del objeto sin sobrescribir todo el campo.
- **Índices sobre columnas generadas:** Cómo indexar un valor específico que vive dentro de un JSON para que las búsquedas sean instantáneas.

```
-- Crear tabla con columna JSON
CREATE TABLE productos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(100),
  especificaciones JSON
);

-- Insertar datos
INSERT INTO productos (nombre, especificaciones) VALUES
('Laptop Pro', '{"marca": "TechCorp", "ram": "16GB", "precio": 1200, "colores": ["negro", "plata"]}'),
('Mouse Gamer', '{"marca": "GameCo", "dpi": 16000, "precio": 45, "inalambrico": true}');

-- JSON_EXTRACT: Obtener valores específicos
SELECT nombre,
       JSON_EXTRACT(especificaciones, '$.marca') AS marca,
       JSON_EXTRACT(especificaciones, '$.precio') AS precio
FROM productos;

-- JSON_INSERT: Añadir nueva propiedad (solo si no existe)
UPDATE productos
SET especificaciones = JSON_INSERT(especificaciones, '$.garantia', '2 años')
WHERE id = 1;

-- JSON_REPLACE: Modificar valor existente
UPDATE productos
SET especificaciones = JSON_REPLACE(especificaciones, '$.precio', 1150)
WHERE nombre = 'Laptop Pro';
```

Ilustración 2 – uso de JSON en MySQL - Fuente: Autor de contenidos

3. Fundamentos de NoSQL y MongoDB

3.1. ¿Qué es NoSQL?

NoSQL (Not Only SQL) es un término que abarca diversos tipos de bases de datos

diseñadas para manejar datos de manera diferente a las bases de datos relacionales tradicionales. Surgieron como respuesta a las limitaciones de las bases de datos SQL cuando se enfrentan a grandes volúmenes de datos, alta velocidad de cambio y estructuras de datos variadas.

Características de bases de datos NoSQL

- **Flexibilidad de esquema:** A diferencia de SQL, donde debes definir tablas y columnas rígidamente, NoSQL permite almacenar datos sin una estructura predefinida. Puedes agregar campos nuevos sobre la marcha sin afectar registros existentes.
- **Escalabilidad horizontal:** NoSQL está diseñado para distribuirse fácilmente en múltiples servidores. En lugar de necesitar un servidor cada vez más potente (escalado vertical), puedes agregar más servidores comunes (escalado horizontal).
- **Alto rendimiento:** Optimizado para operaciones específicas como lecturas/escrituras rápidas, manejo de grandes volúmenes de datos y procesamiento distribuido.
- **Diversos modelos de datos:** NoSQL no es un único tipo de base de datos, sino que incluye varios modelos según tus necesidades de almacenamiento.

3.2. ¿Qué es MongoDB?



Ilustración 3 –MongoDB - Fuente: Autor de contenidos

MongoDB es la base de datos NoSQL más popular. Almacena datos en documentos tipo JSON (técnicamente BSON - Binary JSON) dentro de colecciones.

Cuando la escalabilidad horizontal y la velocidad de escritura son la prioridad, entramos al mundo NoSQL. **MongoDB** destaca como el estándar de la industria para bases de datos orientadas a documentos.

- **Escalabilidad Horizontal (Sharding):** Distribución de datos en varios servidores.
- **Esquema Dinámico:** Capacidad de insertar documentos con diferentes estructuras en la misma colección.

- **Alta Disponibilidad:** Uso de Replica Sets para asegurar que los datos siempre estén disponibles.

3.3. Conceptos fundamentales de MongoDB

- **Documento:** Unidad básica de datos, similar a un objeto JSON. Puede contener campos anidados, arrays y subdocumentos.

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "nombre": "Ana García",
  "edad": 28,
  "email": "ana@example.com",
  "direccion": {
    "calle": "Av. Principal 123",
    "ciudad": "San Salvador"
  },
  "intereses": ["programación", "música", "viajes"]
}
```

Ilustración 4 –

Documento en MongoDB - Fuente: Autor de contenidos

- **Colección:** Grupo de documentos, equivalente a una tabla en SQL, pero sin esquema fijo.

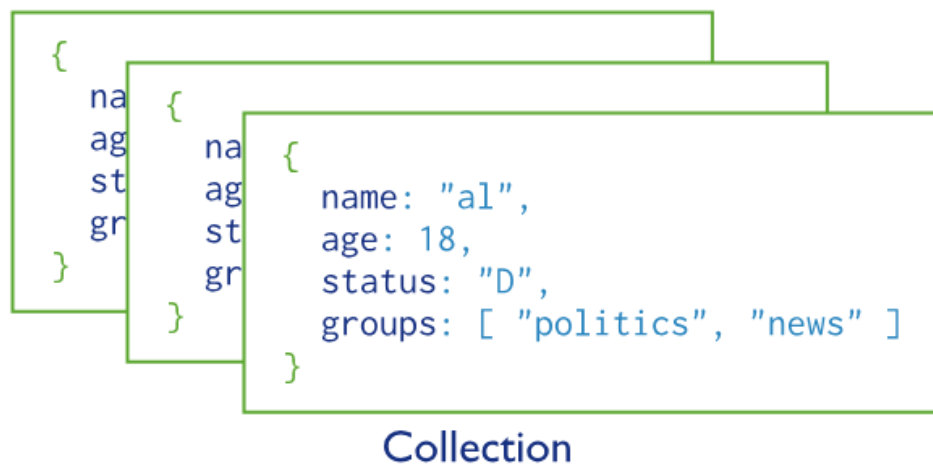


Ilustración 5 – Colecciones en MongoDB - Fuente: Autor de contenidos

- **Base de datos:** Contiene múltiples colecciones.

3.4. Ventajas de MongoDB

MongoDB brilla en escenarios donde necesitas flexibilidad en la estructura de datos. Si tu aplicación maneja diferentes tipos de entidades o los requerimientos cambian frecuentemente, MongoDB te permite adaptar el esquema sin migraciones complejas. Es especialmente útil para aplicaciones web modernas, APIs REST, sistemas de gestión de contenido y aplicaciones que requieren escalabilidad.

3.5. Operaciones básicas

Las operaciones CRUD en MongoDB son intuitivas y se alinean bien con el desarrollo moderno:

- **Insertar:** Puedes insertar uno o múltiples documentos con `insertOne()` o `insertMany()`.
- **Consultar:** El método `find()` acepta filtros flexibles y puede proyectar solo los campos que necesitas.
- **Actualizar:** Con `updateOne()` o `updateMany()` puedes modificar documentos existentes, incluso campos anidados.
- **Eliminar:** `deleteOne()` y `deleteMany()` remueven documentos según criterios específicos.

4. Cuando usar MongoDB vs SQL

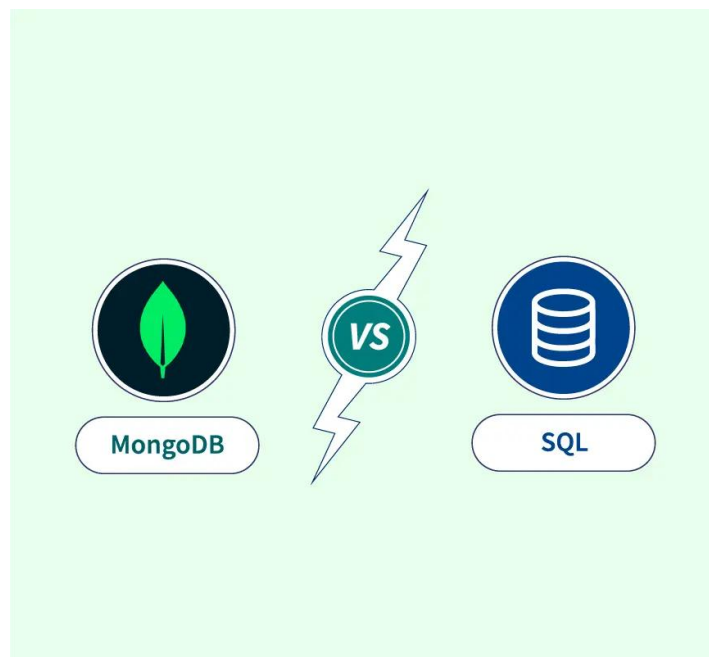


Ilustración 5 –MongoDB vs SQL - Fuente: Autor de contenidos

MongoDB es ideal cuando necesitas desarrollo ágil, manejas datos semi-estructurados o no estructurados, requieres escalabilidad horizontal, o trabajas con grandes volúmenes de datos de lectura/escritura. Las bases de datos SQL siguen siendo superiores para transacciones complejas que requieren integridad ACID estricta, relaciones muy complejas entre múltiples entidades, o cuando necesitas consultas analíticas complejas con múltiples JOINS.

4.1. **Consideraciones importantes**

Aunque MongoDB es flexible y poderoso, requiere pensar diferente sobre el diseño de datos. En lugar de normalizar como en SQL, a menudo es mejor desnormalizar y duplicar datos cuando tiene sentido para tu patrón de consultas. La regla general es: diseña tu esquema basándote en cómo tu aplicación accederá a los datos, no en cómo los datos se relacionan conceptualmente.

Recursos de Apoyo



Material de Lectura

[Uso de JSON en MySQL](#)



Guía

[Guía de cómo hacer particiones en MySQL](#)



Video

[Curso corto de MongoDB](#)